Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science andCommerce

Department of Information Technology (B.Sc.I.T Semester V)

AI

Practical – VII

Roll No.: T003	Name: Pranay Bhadwalkar
Class: TYIT	Batch: B1
Date of Assignment: 01/08/2026	Date/Time of Submission:01/08/2026

Machine Learning

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

a. Load dataset using Pandas.

b. Perform basic preprocessing (handling missing values or scaling).

c. Split dataset into training and testing sets.

d. Train a simple classifier.

e. Display training and testing accuracy.

DATASET - IRIS

```
pandas as pd
import numpy as np
from sklearn.preprocessing iimport mport LabelEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
```

- Load the Dataset

```
# 1. Load the Dataset
df = pd.read_csv("iris - iris - iris - iris.csv")
print("========== Dataset Loaded ==========\n")
```

- Display the Dataset

```
# 2. Display the Dataset
print("First 5 Records:")
print(df.head())
```

```
First 5 Records:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

- Check Dataset Information

```
# 3. Check Dataset Information
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
```

- Display Dataset Shape

```
# 4. Display Dataset Shape
print("\nDataset Shape:")
print(df.shape)
```

```
Dataset Shape:
(150, 5)
```

- Display Column Names

```
# 5. Display Column Names
print("\nColumn Names:")
print(df.columns)
```

```
Column Names:
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
       'species'],
      dtype='str')
```

- Display Summary Statistics

```
# 6. Display Summary Statistics
print("\nSummary Statistics:")
print(df.describe())
```

```
Summary Statistics:
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.054000      3.758667     1.198667
std        0.828066     0.433594      1.764420     0.763161
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

- Check Data Types

```
# 7. Check Data Types
print("\nData Types:")
print(df.dtypes)
```

```
Data Types:
sepal_length    float64
sepal_width     float64
petal_length    float64
petal_width     float64
species             str
dtype: object
```

- Check Missing Values

```
# 8. Check Missing Values
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

- Handle Missing Values

```
# 9. Handle Missing Values
numeric_columns = df.select_dtypes(include=np.number).columns
imputer = SimpleImputer(strategy="mean")
df[numeric_columns] = imputer.fit_transform(df[numeric_columns])
print("\nMissing Values After Handling:")
print(df.isnull().sum())
```

```
Missing Values After Handling:
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

- Remove Duplicate Records

```
# 10. Remove Duplicate Records
print("\nDuplicate Records Before Removing:")
print(df.duplicated().sum())
df = df.drop_duplicates()
print("Duplicate Records After Removing:")
print(df.duplicated().sum())
```

```
Duplicate Records Before Removing:
3
Duplicate Records After Removing:
0
```

- Encode Categorical Data (Label Encoding / One-Hot Encoding)

```
# 11. Encode Categorical Data
encoder = LabelEncoder()
df["species"] = encoder.fit_transform(df["species"])
print("\nEncoded Dataset:")
print(df.head())
```

```
Encoded Dataset:
   sepal_length  sepal_width  petal_length  petal_width  species
0           5.1          3.5           1.4          0.2        0
1           4.9          3.0           1.4          0.2        0
2           4.7          3.2           1.3          0.2        0
3           4.6          3.1           1.5          0.2        0
4           5.0          3.6           1.4          0.2        0
```

- Rename Columns

```
# 12. Rename Columns
df.rename(columns={
   "sepal_length":"Sepal_Length",
   "sepal_width":"Sepal_Width",
   "petal_length":"Petal_Length",
   "petal_width":"Petal_Width",
   "species":"Species"
}, inplace=True)
print("\nColumn Names After Rename:")
```

print(df.columns)

```
Column Names After Rename:
Index(['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width',
       'Species'],
      dtype='str')
```

- Drop Unnecessary Columns

```
# 13. Drop Unnecessary Columns
# Iris dataset has no unnecessary columns.
# Example (if any column existed):
# df.drop(columns=["ID"], inplace=True)
print("\nNo unnecessary columns to drop.")
```

```
No unnecessary columns to drop.
```

- Select Features (Feature Selection)

```
# 14. Select Features
features = [
   "Sepal_Length",
   "Sepal_Width",
   "Petal_Length",
   "Petal_Width"
]
print("\nSelected Features:")
print(features)
```

```
Selected Features:
['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width']
```

- Create New Features (Feature Engineering)

```
# 15. Create New Features
df["Petal_Area"] = df["Petal_Length"] * df["Petal_Width"]
print("\nNew Feature Created: Petal_Area")
print(df.head())
```

```
New Feature Created: Petal_Area
   Sepal_Length  Sepal_Width  Petal_Length  Petal_Width  Species  Petal_Area
0           5.1          3.5           1.4          0.2        0        0.28
1           4.9          3.0           1.4          0.2        0        0.28
2           4.7          3.2           1.3          0.2        0        0.26
3           4.6          3.1           1.5          0.2        0        0.30
4           5.0          3.6           1.4          0.2        0        0.28
```

- Convert Data Types

```
# 16. Convert Data Types
df["Species"] = df["Species"].astype(int)
print("\nData Types After Conversion:")
print(df.dtypes)
```

```
Data Types After Conversion:
Sepal_Length    float64
Sepal_Width     float64
Petal_Length    float64
Petal_Width     float64
Species           int64
Petal_Area      float64
dtype: object
```

- Normalize Data (Min-Max Scaling)

```
# 17. Normalize Data (Min-Max Scaling)
minmax = MinMaxScaler()
columns_to_scale = features + ["Petal_Area"]
df[columns_to_scale] = minmax.fit_transform(df[columns_to_scale])
print("\nNormalized Data:")
print(df.head())
```

```
Normalized Data:
   Sepal_Length  Sepal_Width  Petal_Length  Petal_Width  Species  Petal_Area
0      0.222222     0.625000      0.067797     0.041667        0    0.010787
1      0.166667     0.416667      0.067797     0.041667        0    0.010787
2      0.111111     0.500000      0.050847     0.041667        0    0.009518
3      0.083333     0.458333      0.084746     0.041667        0    0.012056
4      0.194444     0.666667      0.067797     0.041667        0    0.010787
```

- Standardize Data (Standard Scaling)

```
# 18. Standardize Data (Standard Scaling)
standard = StandardScaler()
df[columns_to_scale] = standard.fit_transform(df[columns_to_scale])
print("\nStandardized Data:")
print(df.head())
```

```
Standardized Data:
   Sepal_Length  Sepal_Width  Petal_Length  Petal_Width  Species  Petal_Area
0     -0.915509     1.019971     -1.357737      -1.3357        0   -1.186654
1     -1.157560    -0.128082     -1.357737      -1.3357        0   -1.186654
2     -1.399610     0.331139     -1.414778      -1.3357        0   -1.190920
3     -1.520635     0.101529     -1.300696      -1.3357        0   -1.182388
4     -1.036535     1.249582     -1.357737      -1.3357        0   -1.186654
```

- Split Features and Target Variable

```
# 19. Split Features and Target Variable
X = df[columns_to_scale]
y = df["Species"]
print("\nFeatures (X):")
print(X.head())
print("\nTarget Variable (y):")
print(y.head())
```

```
Features (X):
   Sepal_Length  Sepal_Width  Petal_Length  Petal_Width  Petal_Area
0     -0.915509     1.019971     -1.357737      -1.3357   -1.186654
1     -1.157560    -0.128082     -1.357737      -1.3357   -1.186654
2     -1.399610     0.331139     -1.414778      -1.3357   -1.190920
3     -1.520635     0.101529     -1.300696      -1.3357   -1.182388
4     -1.036535     1.249582     -1.357737      -1.3357   -1.186654

Target Variable (y):
0    0
1    0
2    0
3    0
4    0
Name: Species, dtype: int64
```

DATASET - PENGUINS

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
```

- Load the Dataset

```
# 1. Load the Dataset
df = pd.read_csv("penguins.csv")
print("========== Dataset Loaded ==========\n")
```

```
========== Dataset Loaded ==========
```

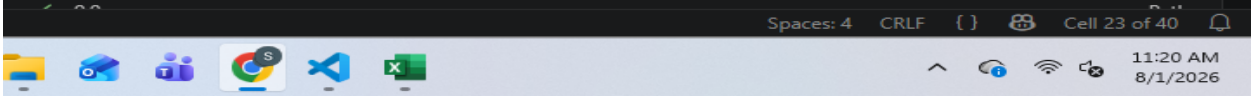
- Display the Dataset

```
# 2. Display the Dataset
print("First 5 Records:")
print(df.head())
```

```
First 5 Records:
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen            39.1           18.7              181.0
1  Adelie  Torgersen            39.5           17.4              186.0
2  Adelie  Torgersen            40.3           18.0              195.0
3  Adelie  Torgersen             NaN            NaN                NaN
4  Adelie  Torgersen            36.7           19.3              193.0

   body_mass_g     sex
0       3750.0    male
1       3800.0  female
2       3250.0  female
3          NaN     NaN
4       3450.0  female
```

```
    # 3. Check Dataset Information
```



- Check Dataset Information

```
# 3. Check Dataset Information
```

```
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    str
 1   island             344 non-null    str
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    str
dtypes: float64(4), str(3)
memory usage: 18.9 KB
```

- Display Dataset Shape

```
# 4. Display Dataset Shape
print("\nDataset Shape:")
print(df.shape)
```

```
Dataset Shape:
(344, 7)
```

- Display Column Names

```
# 5. Display Column Names
print("\nColumn Names:")
print(df.columns)
```

```
Column Names:
Index(['species', 'island', 'bill_length_mm', 'bill_depth_mm',
       'flipper_length_mm', 'body_mass_g', 'sex'],
      dtype='str')
```

- Display Summary Statistics

```
# 6. Display Summary Statistics
print("\nSummary Statistics:")
print(df.describe())
```

```
Summary Statistics:
       bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g
count      342.000000     342.000000         342.000000   342.000000
mean        43.921930      17.151170         200.915205  4201.754386
std          5.459584       1.974793          14.061714   801.954536
min         32.100000      13.100000         172.000000  2700.000000
25%         39.225000      15.600000         190.000000  3550.000000
50%         44.450000      17.300000         197.000000  4050.000000
75%         48.500000      18.700000         213.000000  4750.000000
max         59.600000      21.500000         231.000000  6300.000000
```

- Check Data Types

```
# 7. Check Data Types
print("\nData Types:")
print(df.dtypes)
```

```
Data Types:
species                 str
island                  str
bill_length_mm      float64
bill_depth_mm       float64
flipper_length_mm   float64
body_mass_g         float64
sex                     str
dtype: object
```

- Check Missing Values

```
# 8. Check Missing Values
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
species             0
island              0
bill_length_mm      2
bill_depth_mm       2
flipper_length_mm   2
body_mass_g         2
sex                11
dtype: int64
```

- Handle Missing Values

```
# 9. Handle Missing Values
numeric_columns = df.select_dtypes(include=np.number).columns
imputer = SimpleImputer(strategy="mean")
df[numeric_columns] = imputer.fit_transform(df[numeric_columns])
categorical_columns = df.select_dtypes(include="object").columns
for col in categorical_columns:
    df[col].fillna(df[col].mode()[0], inplace=True)
print("\nMissing Values After Handling:")
print(df.isnull().sum())
```

```
Missing Values After Handling:
species             0
island              0
bill_length_mm      0
bill_depth_mm       0
flipper_length_mm   0
body_mass_g         0
sex                11
dtype: int64
```

- Remove Duplicate Records

```
# 10. Remove Duplicate Records
print("\nDuplicate Records Before Removing:")
print(df.duplicated().sum())
df = df.drop_duplicates()
print("Duplicate Records After Removing:")
print(df.duplicated().sum())
```

```
Duplicate Records Before Removing:
0
Duplicate Records After Removing:
0
```

- Encode Categorical Data (Label Encoding / One-Hot Encoding)

```
# 11. Encode Categorical Data
encoder = LabelEncoder()
df["species"] = encoder.fit_transform(df["species"])
df["island"] = encoder.fit_transform(df["island"])
df["sex"] = encoder.fit_transform(df["sex"])
print("\nEncoded Dataset:")
print(df.head())
```

```
Encoded Dataset:
   species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0        0       2        39.10000       18.70000         181.000000
1        0       2        39.50000       17.40000         186.000000
2        0       2        40.30000       18.00000         195.000000
3        0       2        43.92193       17.15117         200.915205
4        0       2        36.70000       19.30000         193.000000

   body_mass_g  sex
0  3750.000000    1
1  3800.000000    0
2  3250.000000    0
3  4201.754386    2
4  3450.000000    0
```

- Rename Columns

```
# 12. Rename Columns
df.rename(columns={
   "bill_length_mm":"Bill_Length",
   "bill_depth_mm":"Bill_Depth",
   "flipper_length_mm":"Flipper_Length",
   "body_mass_g":"Body_Mass",
   "species":"Species",
   "island":"Island",
   "sex":"Sex"
```

```
}, inplace=True)
print("\nColumn Names After Rename:")
print(df.columns)
```

```
Column Names After Rename:
Index(['Species', 'Island', 'Bill_Length', 'Bill_Depth', 'Flipper_Length',
       'Body_Mass', 'Sex'],
      dtype='str')
```

- Drop Unnecessary Columns

```
# 13. Drop Unnecessary Columns
if "year" in df.columns:
   df.drop(columns=["year"], inplace=True)
print("\nRemaining Columns:")
print(df.columns)
```

```
Remaining Columns:
Index(['Species', 'Island', 'Bill_Length', 'Bill_Depth', 'Flipper_Length',
       'Body_Mass', 'Sex'],
      dtype='str')
```

- Select Features (Feature Selection)

```
# 14. Select Features
features = [
   "Bill_Length",
   "Bill_Depth",
   "Flipper_Length",
   "Body_Mass"
]
print("\nSelected Features:")
print(features)
```

```
Selected Features:
['Bill_Length', 'Bill_Depth', 'Flipper_Length', 'Body_Mass']
```

- Create New Features (Feature Engineering)

```
# 15. Create New Feature
df["Bill_Ratio"] = df["Bill_Length"] / df["Bill_Depth"]
print("\nNew Feature Created: Bill_Ratio")
print(df.head())
```

```
New Feature Created: Bill_Ratio
   Species  Island   Bill_Length    Bill_Depth  Flipper_Length     Body_Mass  \
0        0       2 -8.870812e-01  7.877425e-01   -1.422488e+00 -5.657892e-01
1        0       2 -8.134940e-01  1.265563e-01   -1.065352e+00 -5.031679e-01
2        0       2 -6.663195e-01  4.317192e-01   -4.225067e-01 -1.192003e+00
3        0       2 -5.616754e-16 -9.486367e-16    1.169677e-15  5.005702e-16
4        0       2 -1.328605e+00  1.092905e+00   -5.653611e-01 -9.415172e-01

   Sex  Bill_Ratio
0    1   -1.126106
1    0   -6.427920
2    0   -1.543410
3    2    0.592087
4    0   -1.215663
```

- Convert Data Types

```
# 16. Convert Data Types
df["Species"] = df["Species"].astype(int)
df["Island"] = df["Island"].astype(int)
df["Sex"] = df["Sex"].astype(int)
print("\nData Types After Conversion:")
print(df.dtypes)
```

```
Data Types After Conversion:
Species             int64
Island              int64
Bill_Length       float64
Bill_Depth        float64
Flipper_Length    float64
Body_Mass         float64
Sex                 int64
Bill_Ratio        float64
dtype: object
```

- Normalize Data (Min-Max Scaling)

```
# 17. Normalize Data
columns_to_scale = features + ["Bill_Ratio"]
minmax = MinMaxScaler()
df[columns_to_scale] = minmax.fit_transform(df[columns_to_scale])
print("\nNormalized Data:")
print(df.head())
```

```
Normalized Data:
   Species  Island  Bill_Length  Bill_Depth  Flipper_Length  Body_Mass  Sex  \
0        0       2     0.254545    0.666667        0.152542   0.291667    1
1        0       2     0.269091    0.511905        0.237288   0.305556    0
2        0       2     0.298182    0.583333        0.389831   0.152778    0
3        0       2     0.429888    0.482282        0.490088   0.417154    2
4        0       2     0.167273    0.738095        0.355932   0.208333    0

   Bill_Ratio
0    0.228651
1    0.319487
2    0.303659
3    0.466864
4    0.132672
```

- Standardize Data (Standard Scaling)

```
# 18. Standardize Data
standard = StandardScaler()
df[columns_to_scale] = standard.fit_transform(df[columns_to_scale])
print("\nStandardized Data:")
print(df.head())
```

```
Standardized Data:
   Species  Island   Bill_Length    Bill_Depth  Flipper_Length     Body_Mass  \
0        0       2 -8.870812e-01  7.877425e-01   -1.422488e+00 -5.657892e-01
1        0       2 -8.134940e-01  1.265563e-01   -1.065352e+00 -5.031679e-01
2        0       2 -6.663195e-01  4.317192e-01   -4.225067e-01 -1.192003e+00
3        0       2 -5.616754e-16 -9.486367e-16    1.169677e-15  5.005702e-16
4        0       2 -1.328605e+00  1.092905e+00   -5.653611e-01 -9.415172e-01

   Sex  Bill_Ratio
0    1   -1.038905
1    0   -0.677028
2    0   -0.740084
3    2   -0.089895
4    0   -1.421275
```

- Split Features and Target Variable

```
# 19. Split Features and Target
X = df[columns_to_scale]
y = df["Species"]
print("\nFeatures (X):")
print(X.head())
print("\nTarget Variable (y):")
print(y.head())
```

```
Features (X):
    Bill_Length    Bill_Depth  Flipper_Length     Body_Mass  Bill_Ratio
0 -8.870812e-01  7.877425e-01   -1.422488e+00 -5.657892e-01   -1.038905
1 -8.134940e-01  1.265563e-01   -1.065352e+00 -5.031679e-01   -0.677028
2 -6.663195e-01  4.317192e-01   -4.225067e-01 -1.192003e+00   -0.740084
3 -5.616754e-16 -9.486367e-16    1.169677e-15  5.005702e-16   -0.089895
4 -1.328605e+00  1.092905e+00   -5.653611e-01 -9.415172e-01   -1.421275

Target Variable (y):
0    0
1    0
2    0
3    0
4    0
Name: Species, dtype: int64
```